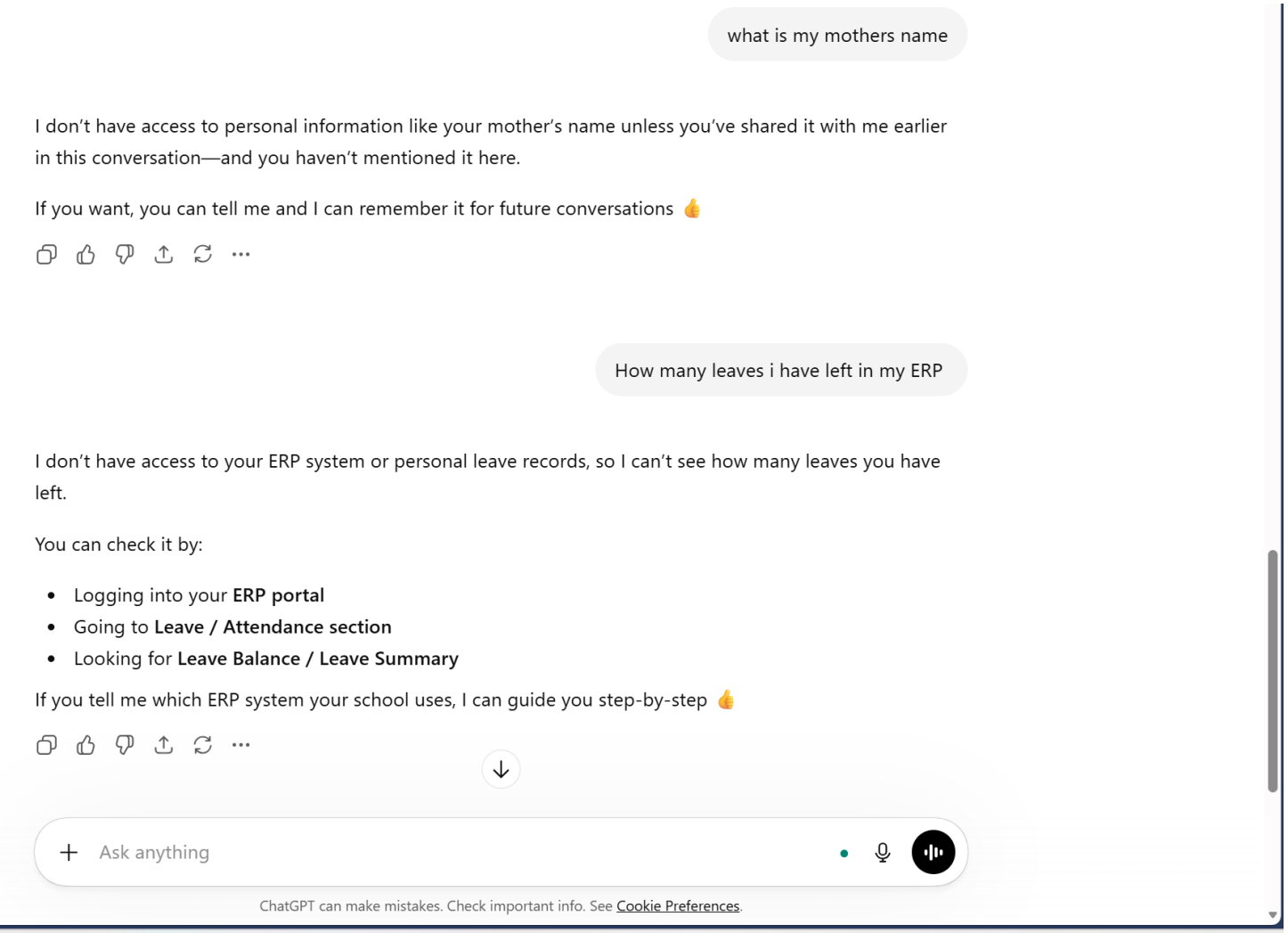
RAG and It’s implementation

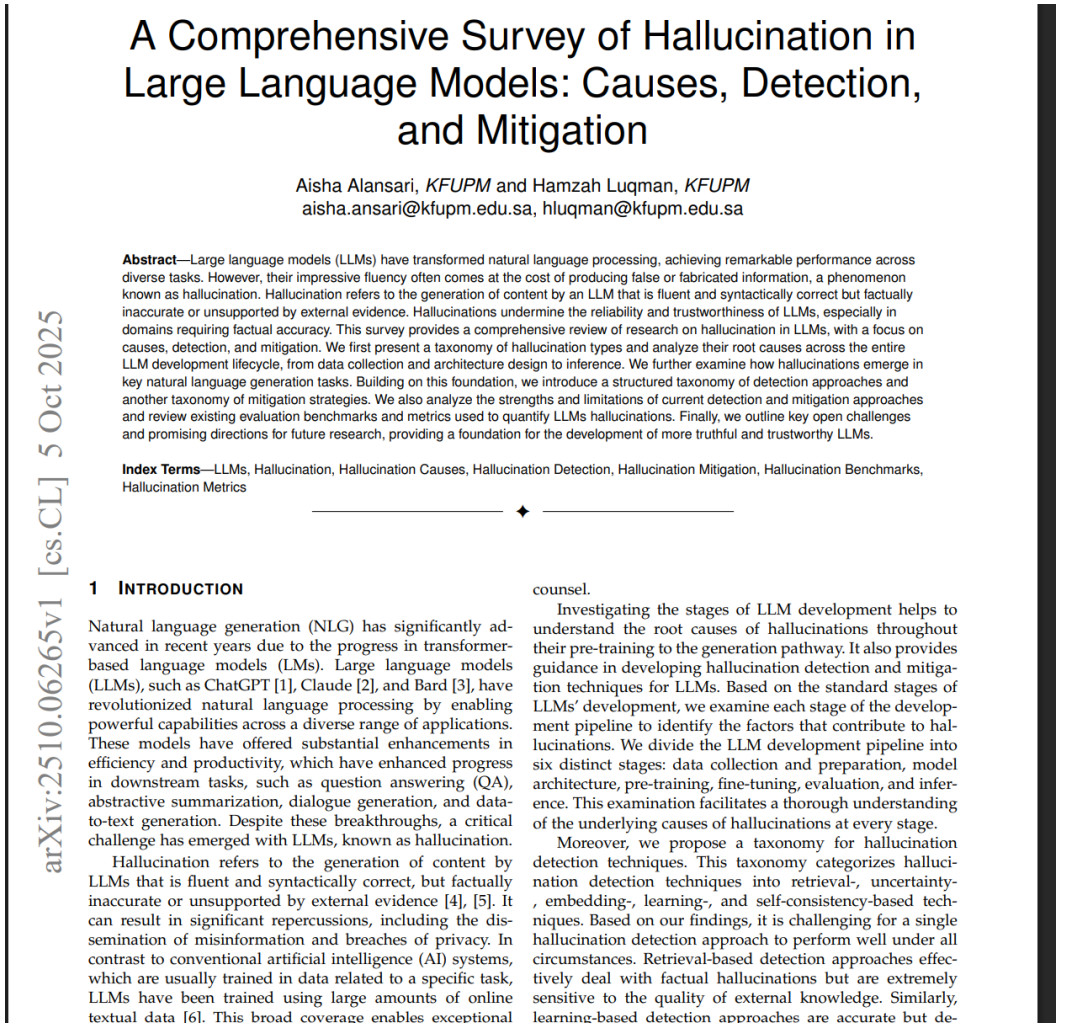
Any GenAI Appication - ChatGPT

https://arxiv.org/pdf/2510.06265v1

LLM Hallucinations

LLM Hallucinations occur when a model generates **incorrect, misleading, or completely fabricated information** that appears believable.





arXiv:2510.06265v1 [cs.CL] 5 Oct 2025

A Comprehensive Survey of Hallucination in Large Language Models: Causes, Detection, and Mitigation

Aisha Alansari, *KFUPM* and Hamzah Luqman, *KFUPM*
aisha.ansari@kfupm.edu.sa, hluqman@kfupm.edu.sa

Abstract—Large language models (LLMs) have transformed natural language processing, achieving remarkable performance across diverse tasks. However, their impressive fluency often comes at the cost of producing false or fabricated information, a phenomenon known as hallucination. Hallucination refers to the generation of content by an LLM that is fluent and syntactically correct but factually inaccurate or unsupported by external evidence. Hallucinations undermine the reliability and trustworthiness of LLMs, especially in domains requiring factual accuracy. This survey provides a comprehensive review of research on hallucination in LLMs, with a focus on causes, detection, and mitigation. We first present a taxonomy of hallucination types and analyze their root causes across the entire LLM development lifecycle, from data collection and architecture design to inference. We further examine how hallucinations emerge in key natural language generation tasks. Building on this foundation, we introduce a structured taxonomy of detection approaches and another taxonomy of mitigation strategies. We also analyze the strengths and limitations of current detection and mitigation approaches and review existing evaluation benchmarks and metrics used to quantify LLMs hallucinations. Finally, we outline key open challenges and promising directions for future research, providing a foundation for the development of more truthful and trustworthy LLMs.

Index Terms—LLMs, Hallucination, Hallucination Causes, Hallucination Detection, Hallucination Mitigation, Hallucination Benchmarks, Hallucination Metrics

1 Introduction

Natural language generation (NLG) has significantly advanced in recent years due to the progress in transformer-based language models (LMs). Large language models (LLMs), such as ChatGPT [1], Claude [2], and Bard [3], have revolutionized natural language processing by enabling powerful capabilities across a diverse range of applications. These models have offered substantial enhancements in efficiency and productivity, which have enhanced progress in downstream tasks, such as question answering (QA), abstractive summarization, dialogue generation, and data-to-text generation. Despite these breakthroughs, a critical challenge has emerged with LLMs, known as hallucination.

Hallucination refers to the generation of content by LLMs that is fluent and syntactically correct, but factually inaccurate or unsupported by external evidence [4], [5]. It can result in significant repercussions, including the dissemination of misinformation and breaches of privacy. In contrast to conventional artificial intelligence (AI) systems, which are usually trained in data related to a specific task, LLMs have been trained using large amounts of online textual data [6]. This broad coverage enables exceptional counsel.

Investigating the stages of LLM development helps to understand the root causes of hallucinations throughout their pre-training to the generation pathway. It also provides guidance in developing hallucination detection and mitigation techniques for LLMs. Based on the standard stages of LLMs' development, we examine each stage of the development pipeline to identify the factors that contribute to hallucinations. We divide the LLM development pipeline into six distinct stages: data collection and preparation, model architecture, pre-training, fine-tuning, evaluation, and inference. This examination facilitates a thorough understanding of the underlying causes of hallucinations at every stage.

Moreover, we propose a taxonomy for hallucination detection techniques. This taxonomy categorizes hallucination detection techniques into retrieval-, uncertainty-, embedding-, learning-, and self-consistency-based techniques. Based on our findings, it is challenging for a single hallucination detection approach to perform well under all circumstances. Retrieval-based detection approaches effectively deal with factual hallucinations but are extremely sensitive to the quality of external knowledge. Similarly, learning-based detection approaches are accurate but de-

Why LLM Hallucinates?

• Knowledge Cutoff

- A **knowledge cutoff** is the point in time after which an LLM has **no awareness of new information**.
- Example:
  If a model is trained up to 2024, it **won't know events from 2025–2026** unless connected to external tools.
- **Why it causes hallucination**
  - When asked about recent events, the model:
    - Tries to **predict a plausible answer**
    - Instead of saying "I don't know"
    - May generate **incorrect or fabricated information**

• Domain Knowledge Gap

- A **domain knowledge gap** occurs when the model has **limited or weak understanding in a specific subject area.**
- Common domains:
  - Medical
  - Legal
  - Highly technical research
  - Niche industry knowledge
- **Why it causes hallucination**
  - Training data may have **less coverage in that domain**
  - Model fills gaps using **general language patterns**
  - Produces **incorrect but convincing explanations**

Why LLM Hallucinates?

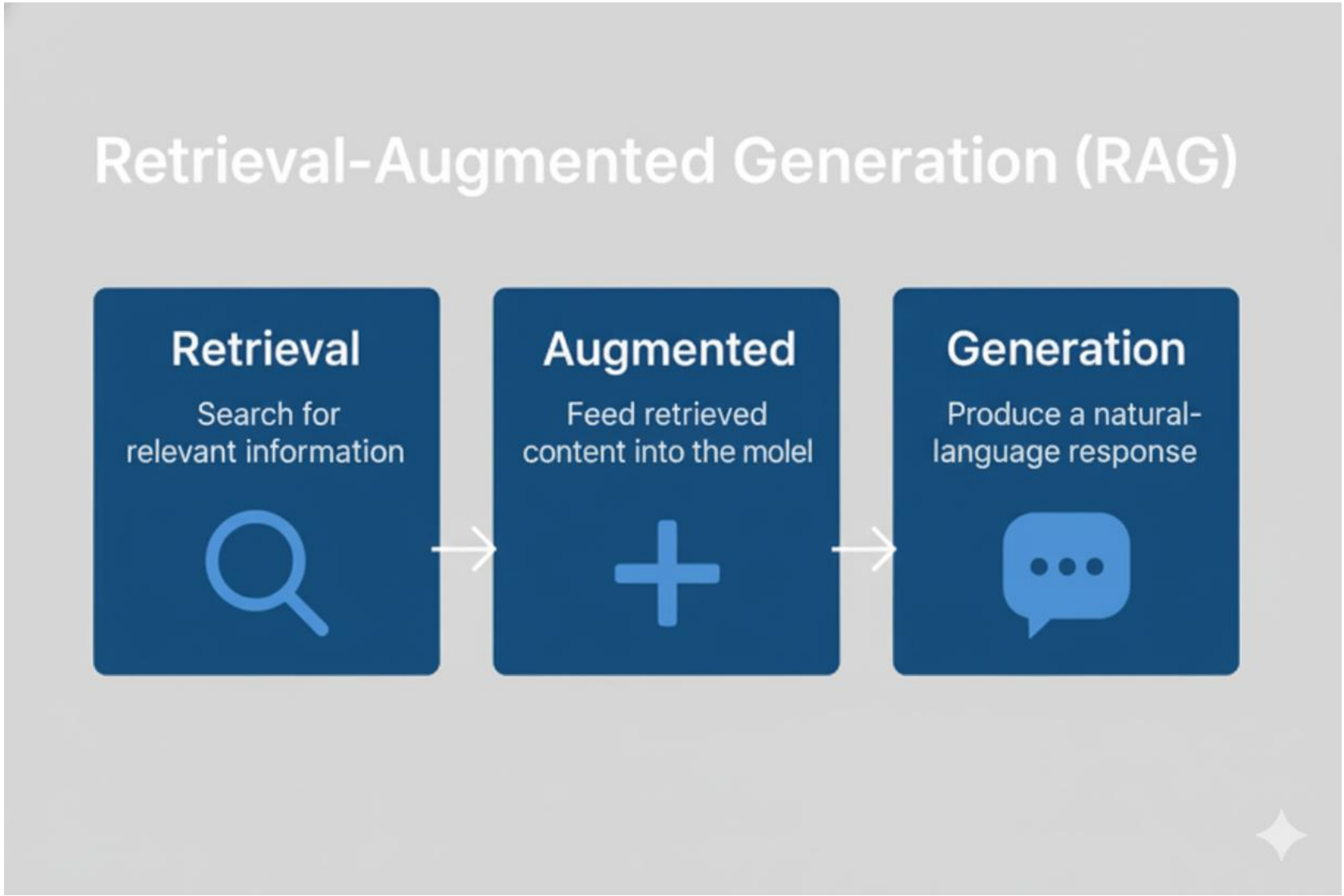
Other Reasons

- **Training Data Limitations**
    - •Models are trained on **imperfect, incomplete, or biased data**
    - •If data has errors → model may reproduce

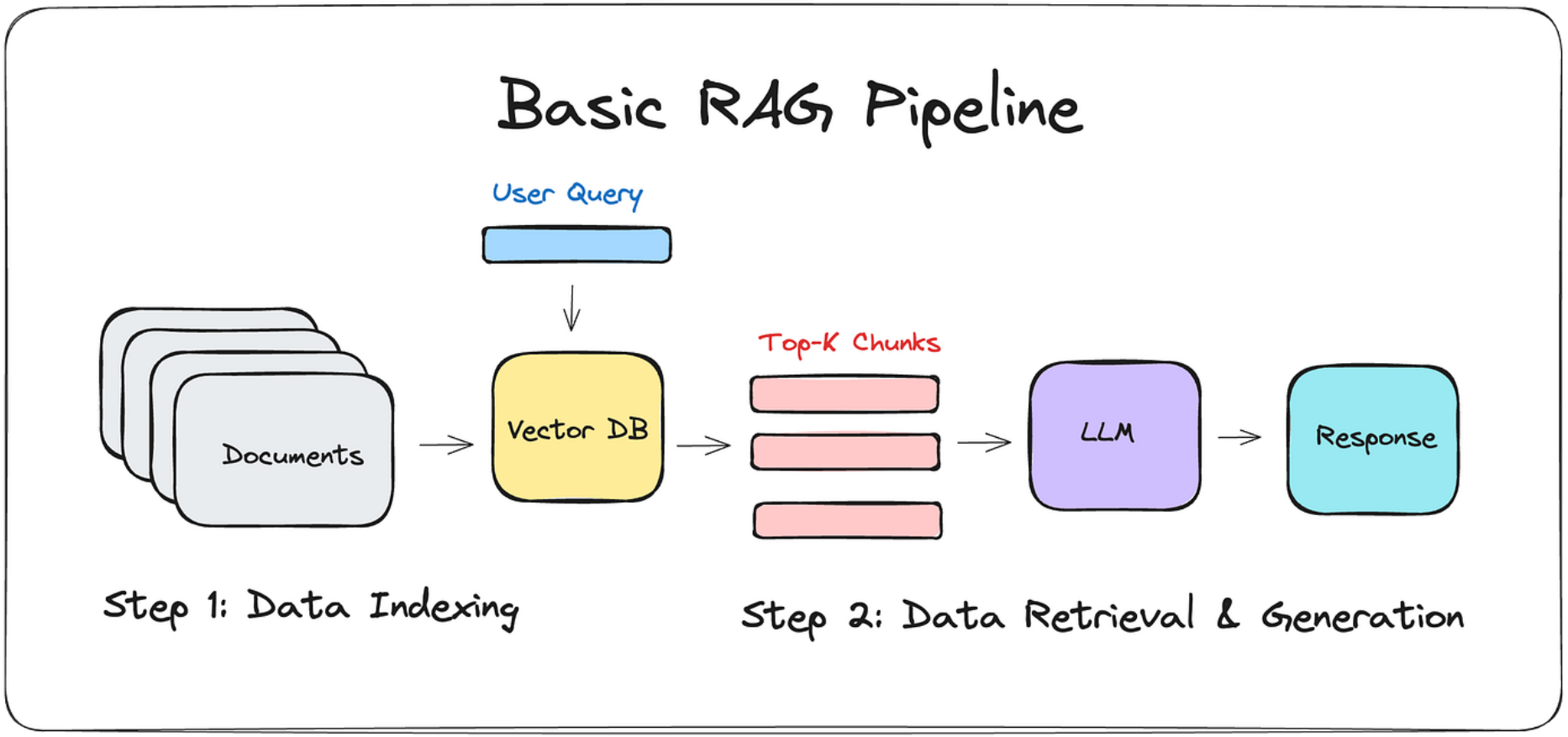
•**Lack of Grounding**

- •Not connected to external databases or verified sources

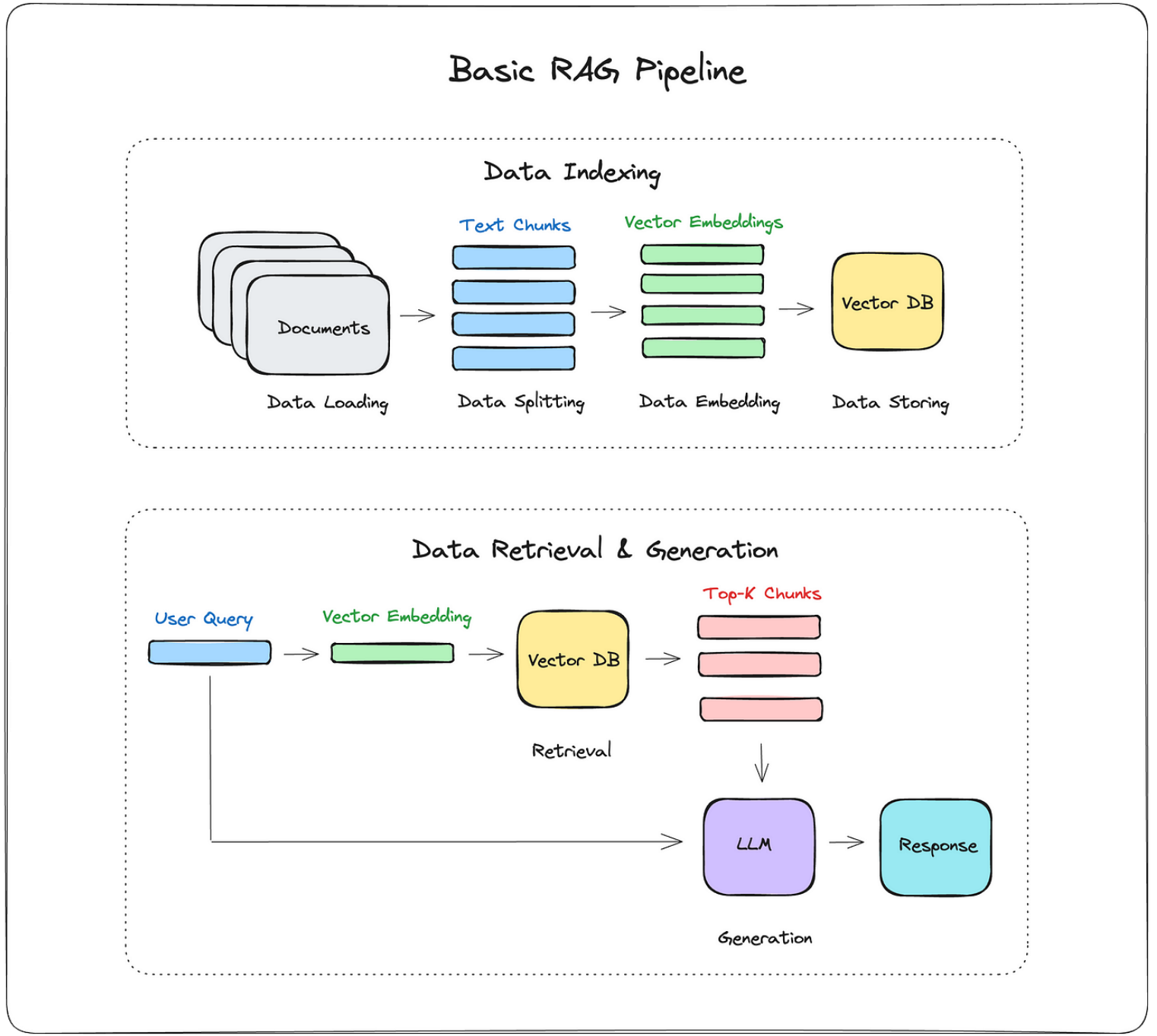
RAG – Retrieval Augmented Generation



RAG Pipeline

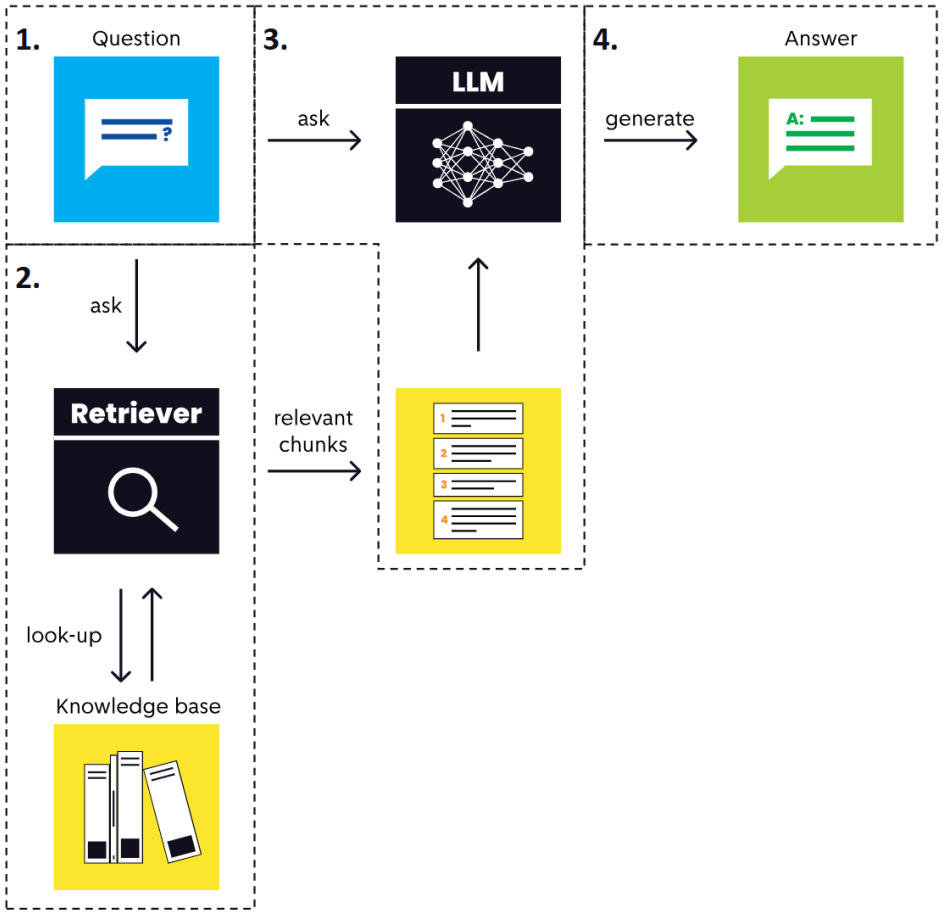


RAG Pipeline



How RAG Works

1. **User Asks a Question:** Example - "Who is the King of The Netherlands?"
2. **Find Relevant Information:** looks for the most relevant document chunks that match the question in the knowledge base.
3. **Combine Question and Information:** The questions and collected document chunks are combined and forwarded to the Generative AI.
4. **Generate an Answer:** Generative AI provides a final answer based on combined question and documents.



LangChain Document Structure

Understanding the core components of LangChain Documents

LangChain Document

```
from langchain.schema import Document
```

Core Components:

- **page_content (str)**
- **metadata (dict)**

```
# Creating a Document
doc = Document(
    page_content=  "RAG is a technique..." ,
    metadata={
        "source" :  "chapter1.pdf" ,
        "page"  :  5 ,
        "timestamp"   :  "2024-01-15"
)  }
```

page_content (String)

The actual text content of the document
- Contains the main information to be embedded and searched
- Must be a string (can be any length)

Examples:

Research Paper:
"Retrieval-Augmented Generation (RAG) combines the benefits of pre-trained language models with information retrieval systems to generate more accurate and contextual responses..."

Product Manual:
"To install the software, first ensure your system meets the minimum requirements: Windows 10 or later, 8GB RAM, and at least 20GB of free disk space..."

Best Practices:
- Keep content focused and coherent
- Remove unnecessary formatting before storage
- Consider chunk size limits (typically 500-2000 tokens)

metadata (Dictionary)

Additional information about the document
- Used for filtering, tracking, and context
- Can contain any JSON-serializable data

Common Metadata Fields:

source
File path or URL
`"docs/manual.pdf"`

page / chunk_id
Location in document
`page: 42, chunk: 7`

timestamp
Creation/modification date
`"2024-01-15T10:30:00Z"`

author
Document creator
`"John Doe"`

category / type
Document classification
`"technical", "legal"`

language
Content language
`"en", "es", "fr"`

Tip: Add custom fields for your use case
Examples: department, security_level, version, keywords, embeddings_model

LangChain Document Loaders

PDFLoader

```
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("file.pdf")
documents = loader.load()
```

CSVLoader

```
from langchain.document_loaders import CSVLoader
loader = CSVLoader("data.csv")
documents = loader.load()
```

WebBaseLoader

```
from langchain.document_loaders import WebBaseLoader
loader = WebBaseLoader("https://...")
documents = loader.load()
```

DirectoryLoader

```
from langchain.document_loaders import DirectoryLoader
loader = DirectoryLoader("./docs")
documents = loader.load()
```

Additional Loaders:

- UnstructuredLoader (multiple formats)
- JSONLoader
- TextLoader
- GitbookLoader
- NotionDirectoryLoader
- GoogleDriveLoader
- AirtableLoader
- SlackDirectoryLoader
- S3FileLoader
- YouTubeLoader
- WikipediaLoader
- ArxivLoader
- ConfluenceLoader
- DocugamiLoader
- EverNoteLoader
- HuggingFaceDatasetLoader

Document Transformers (Text Splitters)

CharacterTextSplitter

```
splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
```

RecursiveCharacterTextSplitter

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    separators=["\n\n", "\n", " "]
)
```

TokenTextSplitter

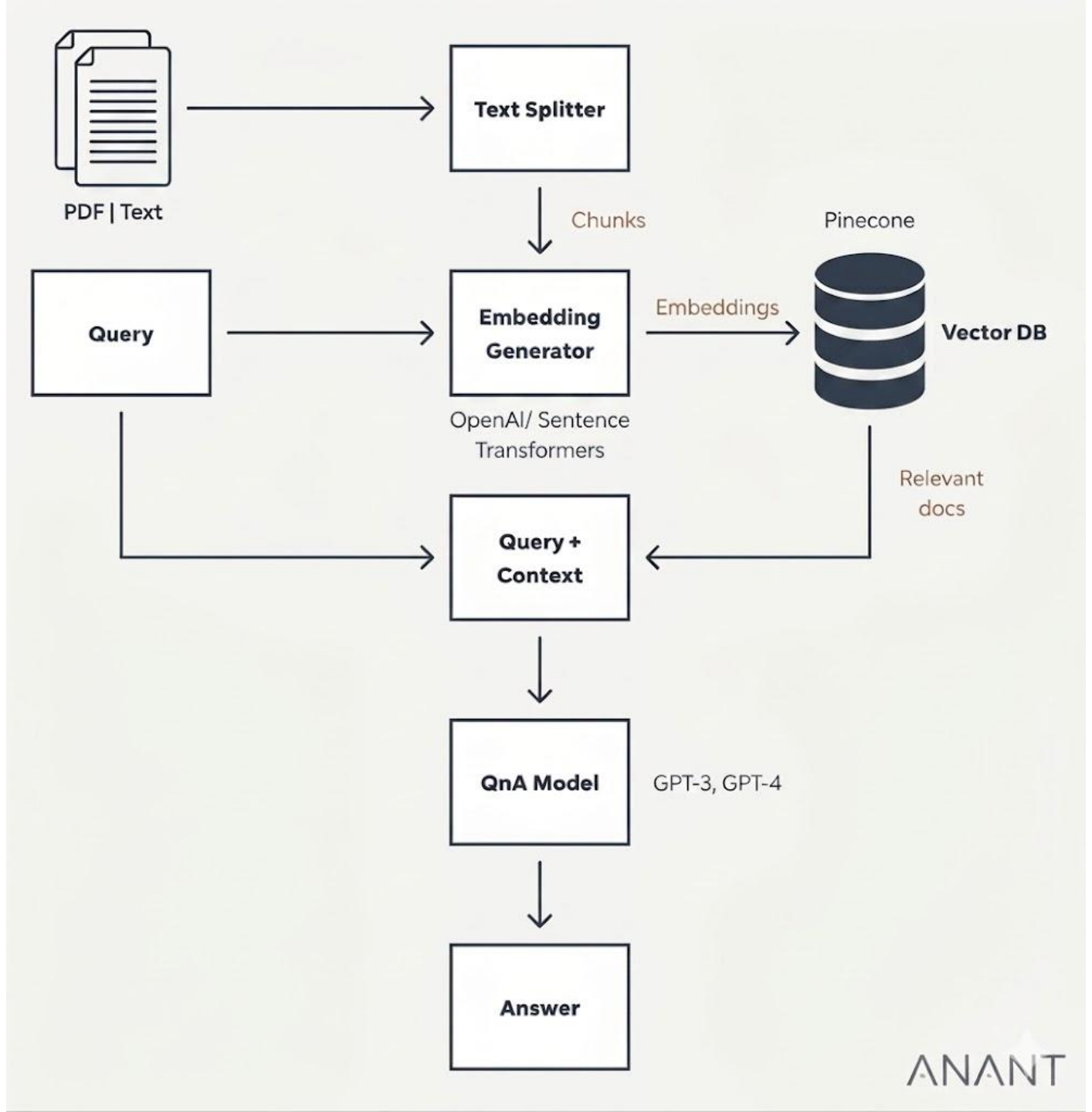
```
splitter = TokenTextSplitter(
    chunk_size=500,
    model_name="gpt-3.5-turbo"
)
```

SemanticChunker

```
splitter = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="percentile"
)
```

```
# Split documents into chunks
chunks = splitter.split_documents(documents)
# Each chunk is a new Document with preserved metadata
```

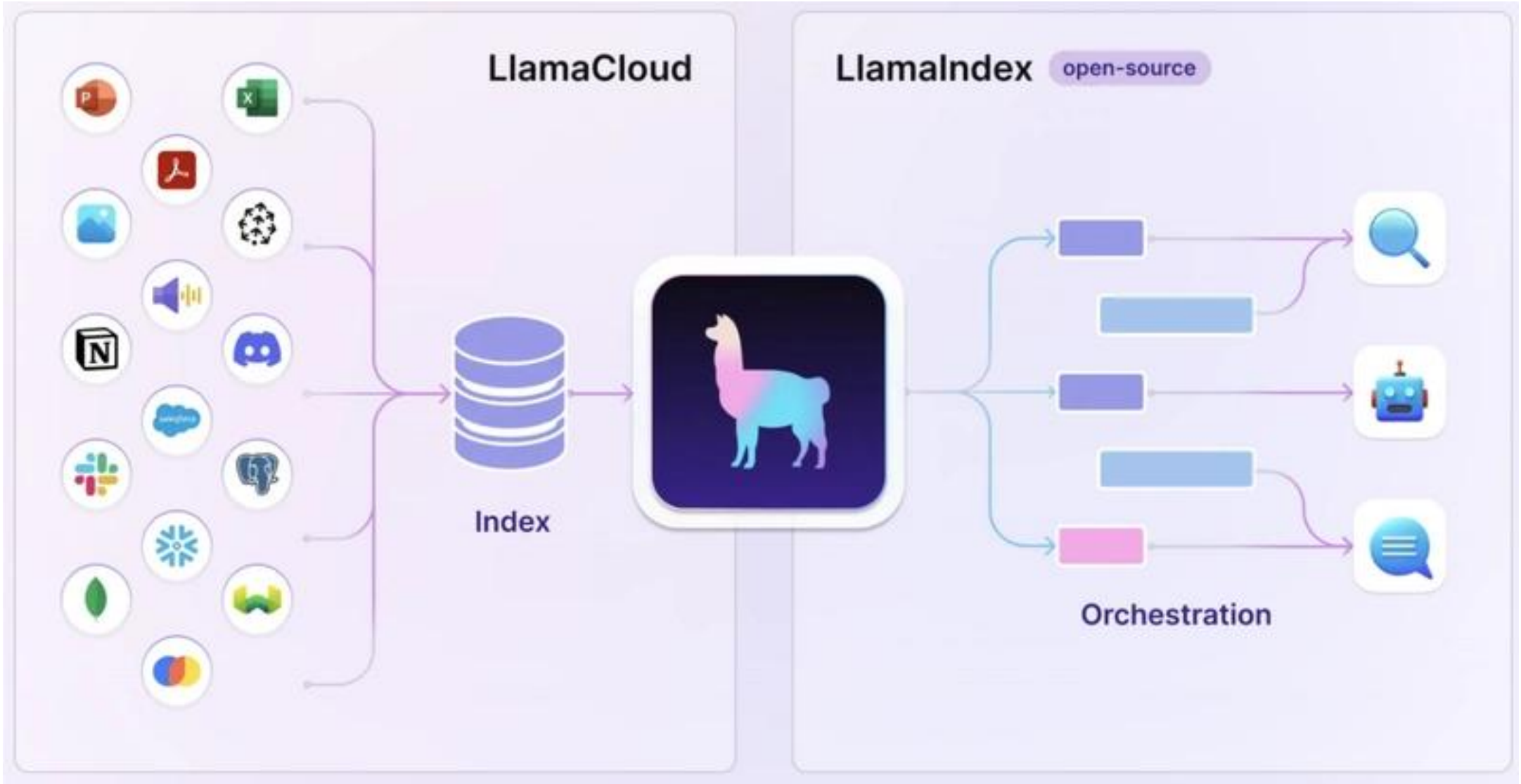
Different Database


PDF | Text
Text Splitter
Chunks
Pinecone
Query
Embedding Generator
Embeddings
Vector DB
OpenAI/ Sentence Transformers
Relevant docs
Query + Context
QnA Model
GPT-3, GPT-4
Answer
ANANT

LlamaIndex

A Toolkit
Designed to Connect
LLM's
with
External Data

A FRAMEWORK FOR CONTEXT-AUGMENTED LLM APPLICATIONS


LlamaCloud
LlamaIndex
open-source
Index
Orchestration

Context

- LLMs are powerful for knowledge generation and reasoning , pre-trained on publicly available data.
- To augment LLMs with private data, in-context learning has emerged as a paradigm, where context is inserted into the input prompt.
- In-context learning takes advantages of LLM's reasoning capabilities to generate a response, offering a simple and effective way to use them for data augmentation.
- And RAG is the solution – In RAG, we store the context in terms of vectors in VDBs – That provide context to the LLM along with the query and augment the intended result.

Context…

- Using Llama Index performing document search is very efficient.
- If you have data in different formats - .pdf or .txt, .csv etc – Llama Index is the best solution.
- Means: Data in any format – Structure, un-structured and semi-structured
- **Steps**:
  - **Data Ingestion** – it provides a lot libraries to connect with external data sources – pdf, csv, Apis, documents, sql etc
  - **Data Indexing** – Store and Index for different use cases
  - **Query Interface** – It provides an interface that accepts any prompt over your data and returns a knowledge augmented response.

Library Installation

- !pip install llama-index

- uv add llama-index

Llama Index vs Langchain

Feature	Llama Index	Langchain
Primary Focus	Intelligent search and data indexing & retrival	Building a wide range of GenAI applications – chain, memory, tools integrations.
Flexibility	Specialized for efficient and fast search	General purpose framework with more flexibility with application behaviour
Data handling	Ingesting, structuring and accessing pvt or domain specific data. - It loads, and the structure and then index. - Best for complex data structure	Loading, processing, and indexing data for various use cases - Loading like Llama index but treat data a langchain document. - Simple document retrieval for chatbot.
Supports LLMs (as per dec 2023)	Connects to any LLM provider like OpenAI, Anthropic, Hugging Face etc	Supports various LLMs including hugging face OpenAI etc
Use Cases	Best for applications that requires quick data look-up and retrieval	Suitable for applications that needs complex interactions like chatbot, FQA, Summarizations etc
Application Breadth	Focus of search centric applications - RAG	Supports a broad rang of applications
Integration	Function as a smart storage mechanism	Designed to bring multiple tools together to chain operations